

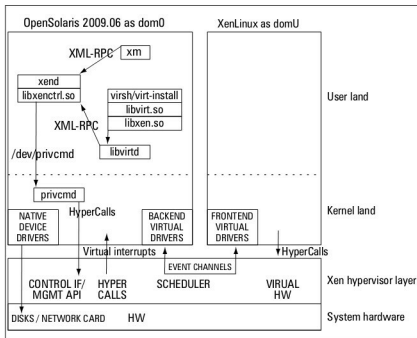


Figure 1: Thin hypervisor sitting on the hardware

The back-end driver, in turn, communicates with the actual driver that controls the device. What that means is, generally, all the peripheral devices are controlled by drivers in dom0, rather than being present in the hypervisor itself. This is a very elegant idea to keep the hypervisor slick and fast. Front-end drivers are class-specific—i.e., there will be one FE for block devices, irrespective of whether the actual device is an IDE, an SCSI or FC. Likewise, there will be only one FE for network devices.

Virtual interrupts: Just as hypercalls provide guest-to-Xen communication, 'event channels' provide Xen-to-guest communication. They are the software equivalent of hardware interrupts. They provide an asynchronous notification mechanism. An event channel is associated with every virtual device. IO request/completion notifications happen via event channels.

Start info page: A PV guest booting in the Xen environment does not have access to the firmware. This is where the 'start info' page kicks in. It's a page mapped into the guest address space by the hypervisor with

details on how much memory the guest is configured with, XenStore details, console details and another page called the 'shared info' page. The `/usr/include/xen/xen.h` file contains the declaration of 'struct start_info'. A guest kernel starts off with information present in this page and then it maps in the XenStore page to get all the devices configured for it. It also gets the *console page* to print out any debug information and thus troubleshoots any bugs present early in the boot phase. A PV guest starts in protected mode unlike a normal OS, which starts in real mode.

Shared info page: Any OS starts off with a single CPU and gets the other CPUs online in an SMP system later in the boot process. Likewise, even a PV guest starts off with a single CPU and the rest of the CPU information is retrieved from the 'shared info' page. It also contains architecture-specific information, if any. Besides this, the 'shared info' page also contains asynchronous notifications pending to the guest as well as those masked by the guest—think of them as signals

pending to a process and those masked by a process. A guest has to explicitly map in the 'shared info' page before it could access the contents.

Balloon thread: This runs in a guest kernel and coordinates with the hypervisor to get more physical memory to a guest; or in the case of overall system pressure, gives back some pages to the hypervisor.

Scheduling: The Xen hypervisor uses the *credit scheduler* to multi-task virtual CPUs onto the physical CPUs. The *credit scheduler* is more like the *fair share scheduler* you will find on OpenSolaris.

Time keeping: When a guest goes off the CPU and comes back later, it has no clue how much *wall clock time* lapsed in between. To handle this, hypervisor stores the *walk clock time* when the guest actually booted in the *shared info page*. The *virtual clock time* of a guest increments while the guest is active. Using these two, the current *wall clock time* is calculated. A PV guest can also sync with a NTP source to keep *wall clock time* accurate.

The discussion thus far is encapsulated in Figure 1. Here, two guests are running simultaneously on the system. One is the privileged guest dom0, which hosts quite a few services for the peripherals, and hosts the back-end drivers. The second is an unprivileged guest that talks to the devices via the front-end drivers.

Hopefully, we are now clear on many of the essential concepts associated with Xen virtualisation technology. In Part 2 of this series (turn to page 69), we move on to setting up OpenSolaris 2009.06 as dom0. 

By: Surya Prakkhi

Surya debugs OpenSolaris kernel, both for living and for passion, and relies a lot on the lifeline called dtrace(1M). He occasionally blogs at blogs.sun.com/sprakkhi.